KNN & PCA Assignment

Theoretical

1 What is K-Nearest Neighbors (KNN) and how does it work
Ans K-Nearest Neighbors (KNN) is a simple, supervised machine learning algorithm used for classification and regression tasks. It works by comparing a new data point to existing labeled data and assigning it a label based on the majority class of its 'k' nearest neighbors. The 'k' value is a user-defined constant that represents the number of closest neighbors to consider.

KNN uses distance metrics like Euclidean, Manhattan, or Minkowski to measure similarity between data points. It is a lazy learner, meaning it doesn't learn a model during training but rather stores the entire dataset. During prediction, it calculates the distance from the new point to all others, selects the k closest, and predicts the output based on those.

KNN is intuitive and effective for small datasets but can be computationally expensive and sensitive to irrelevant features or large dataset

2 What is the difference between KNN Classification and KNN Regression

Ans The key difference between KNN Classification and KNN Regression lies in the type of output they predict:

1. KNN Classification is used when the output is categorical (e.g., "spam" or "not spam"). It assigns the new data point to the majority class among its k nearest neighbors.

2. KNN Regression is used when the output is continuous (e.g., price, temperature). It predicts the value by taking the average (or weighted average) of the values of its k nearest neighbors.

3. In classification, the decision is based on voting, while in regression, it's based on averaging.

4. Example: For predicting if a fruit is an apple or orange → classification; for predicting a house price → regression.

5. Both use the same distance metrics and algorithm logic but differ in how the final prediction is calculated.

3 What is the role of the distance metric in KNN?
Ans The distance metric plays a crucial role in K-Nearest Neighbors (KNN) by determining how similarity is measured between data points. It helps identify the 'k' closest neighbors to a new data point.

Here's why it's important:

1. It calculates the closeness between the new point and existing data points.

2. Common distance metrics include Euclidean, Manhattan, and Minkowski distances.

3. The choice of metric can affect model accuracy, especially if features are on different scales.

4. A good metric ensures that similar data points are grouped together, leading to better predictions.

5. If the wrong metric is chosen, KNN might consider irrelevant points as neighbors, causing poor results.

In short, the distance metric directly influences which neighbors are chosen and, therefore, the final prediction.

4 What is the Curse of Dimensionality in KNN

Ans The Curse of Dimensionality in KNN refers to the problems that arise when data has too many features (dimensions).
Here's what it means in simple terms:

1. As the number of dimensions increases, data points become sparser and more spread out.

2. The distance between points becomes less meaningful, making it hard for KNN to find truly "close" neighbors.

3. It reduces the effectiveness of distance metrics, which are crucial for KNN.

4. This can lead to lower accuracy and overfitting.

5. High dimensions also increase computation time and memory usage.

To deal with this, techniques like feature selection or dimensionality reduction (e.g., PCA) are often used before applying KNN.

5 How can we choose the best value of K in KNN

Ans Choosing the best value of K in KNN is important for getting accurate predictions. Here's how you can do it:

1. Try different K values and evaluate performance using a validation set or cross-validation.
2. Use Odd values of K (like 3, 5, 7) to avoid ties in classification problems.
3. A small K (e.g., K=1) can be sensitive to noise and may overfit.
4. A large K smooths out predictions but may underfit by ignoring local patterns.
5. Plot an error rate vs. K graph and choose the K where the error is lowest (elbow method).
6. In regression, use Mean Squared Error (MSE) to compare K values.

In short, there's no fixed rule — the best K depends on the dataset, and it's often found through experimentation and cross-validation.

6 What are KD Tree and Ball Tree in KNN

KD Tree and Ball Tree are data structures used to speed up the KNN algorithm, especially for large datasets.

1. KD Tree (K-Dimensional Tree):

- It's a binary tree that splits data based on feature values at each level.
- Works well for low-dimensional data.
- Helps in efficient nearest neighbor search by eliminating large portions of data.
- Not suitable when the number of dimensions is very high (curse of dimensionality).

2. Ball Tree:

- Organizes data into nested hyperspheres (balls) instead of axis-aligned splits.

- Better for high-dimensional datasets compared to KD Tree.
- It calculates distance from query point to each ball to eliminate distant regions.

Summary:

- Both trees improve KNN performance by reducing the number of distance calculations.
- KD Tree is faster for low dimensions, while Ball Tree handles high dimensions better.

7 When should you use KD Tree vs. Ball Tree

Ans You should choose between KD Tree and Ball Tree based on the dimensionality and distribution of your data:

Use KD Tree when:

1. Data is low-dimensional (typically < 20 dimensions).
2. The data is evenly distributed.
3. You need fast nearest neighbor search for small to medium datasets.

Use Ball Tree when:

1. Data is high-dimensional (typically > 20 dimensions).
2. The data is unevenly distributed or clustered.
3. You need a more flexible structure for computing distances in complex spaces.

Avoid both when:

- The data has very high dimensions (like 100+), where brute-force search may be as efficient due to the curse of dimensionality.

In short:
KD Tree → Low dimensions, faster for simpler data
Ball Tree → Better for high dimensions or clustered data

8 What are the disadvantages of KNN

Ans Here are the main disadvantages of K-Nearest Neighbors (KNN):

1. Computationally expensive: KNN stores all training data and performs distance calculations at prediction time, which can be slow for large datasets.

2. Sensitive to irrelevant features: Unimportant features can distort distance calculations and reduce accuracy.

3. Affected by the curse of dimensionality: In high-dimensional data, distances become less meaningful, making it harder to find true neighbors.

4. No learning phase: KNN is a lazy learner, so it doesn't generalize from training data, leading to slow predictions.

5. Requires proper feature scaling: Features must be normalized (e.g., using Min-Max or StandardScaler), or distance metrics can be misleading.

6. Poor with imbalanced data: KNN might favor the majority class since it relies on neighbor counts.

7. Memory intensive: It needs to store the entire dataset in memory during prediction.

In summary, while KNN is simple and effective for small, well-prepared datasets, it struggles with scalability, speed, and high-dimensional or noisy data.

9 How does feature scaling affect KNN

Ans Feature scaling is crucial in KNN because the algorithm relies on distance metrics (like Euclidean distance) to find the nearest neighbors. Here's how scaling affects KNN:

1. Unequal feature ranges can distort distance calculations — features with larger ranges will dominate the result.

2. For example, if one feature ranges from 0–1000 and another from 0–1, the first will have a much bigger impact on distance.

3. This can lead to incorrect neighbors being chosen, reducing model accuracy.

4. Scaling ensures all features contribute equally to the distance calculation.

5. Common scaling techniques:
    - Min-Max Scaling (values between 0 and 1)
    - Standardization (mean = 0, standard deviation = 1)

In short, without proper feature scaling, KNN might give misleading results, especially when features have different units or magnitudes.

10 What is PCA (Principal Component Analysis)
Ans PCA (Principal Component Analysis) is a dimensionality reduction technique used in data analysis and machine learning.

Here's what it does:

1. PCA transforms high-dimensional data into a lower-dimensional space while keeping the most important information (variance).
2. It finds new axes (called principal components) that are linear combinations of the original features.
3. The first principal component captures the most variance, the second captures the next most, and so on.
4. PCA helps to reduce noise, speed up algorithms, and visualize high-dimensional data.
5. It's commonly used before models like KNN, where too many dimensions can hurt performance (curse of dimensionality).

In simple terms, PCA helps you simplify your dataset without losing too much important information.

11 How does PCA work

Ans PCA (Principal Component Analysis) works by transforming your dataset into a new coordinate system where the axes (called principal components) capture the most variance in the data. Here's a step-by-step explanation:

1. Standardize the data: Scale features to have mean 0 and variance 1 (important if features are on different scales).

2. Compute the covariance matrix: This matrix shows how features vary with each other.

3. Calculate eigenvalues and eigenvectors of the covariance matrix:

    - Eigenvectors define the directions (principal components).

    - Eigenvalues indicate the amount of variance along each principal component.

4. Sort the principal components by their eigenvalues (highest to lowest), so the top components capture the most variance.

5. Select the top 'k' components (based on how much variance you want to retain).

6. Transform the data: Project the original data onto the selected components to get a lower-dimensional version of the data.

In simple terms:

PCA finds the best angles (directions) to look at your data, where it appears most spread out (informative), and removes less important directions, helping in compression, visualization, and reducing noise.

12 What is the geometric intuition behind PCA

Ans The geometric intuition behind PCA (Principal Component Analysis) is about finding the best new axes to view or represent the data in a lower-dimensional space with maximum information (variance).

Here's a simple breakdown:

1. Imagine your data as a cloud of points in space (2D, 3D, or higher).

2. PCA rotates the coordinate system to align it with the directions where the data varies the most.

3. These new directions are called principal components:

    - The 1st principal component is the direction where the data is most spread out (has the highest variance).

    - The 2nd component is orthogonal (at a right angle) to the first and captures the next highest variance, and so on.

4. By projecting the data onto these new axes, you can compress the data into fewer dimensions while keeping the important structure.

Visual example:

In 2D, if data points form an elongated ellipse, PCA will rotate the axes so:

- The x-axis aligns with the longest stretch of the ellipse (1st principal component),

- The y-axis aligns with the narrowest (2nd component, which might be discarded).

So, geometrically, PCA is like reshaping your view of the data to find the flattest angle that still shows the most variation, allowing you to ignore less informative directions.

13 What is the difference between Feature Selection and Feature Extraction

Ans Feature Selection and Feature Extraction are both techniques used for dimensionality reduction, but they work differently.

1. Feature Selection chooses a subset of the original features that are most relevant to the target variable.

2. It removes irrelevant or redundant features without changing the original meaning of data.

3. Common methods include filter, wrapper, and embedded techniques.

4. Example: Selecting top 10 features out of 100 based on correlation or importance.

5. Feature Extraction, on the other hand, creates new features by transforming or combining the original ones.

6. The new features capture the most important information in a different form.

7. Techniques include PCA, t-SNE, and Autoencoders.

8. It’s useful when the original features are highly correlated or not informative on their own.

9. In short, feature selection keeps existing features, while feature extraction creates new ones.

10. Both aim to improve model performance and reduce overfitting by simplifying the dataset.

14 What are Eigenvalues and Eigenvectors in PCA

Ans In PCA (Principal Component Analysis), eigenvalues and eigenvectors are key mathematical concepts used to identify the principal components of the data.

1. Eigenvectors represent the directions or axes along which the data varies the most — these are the principal components.

2. Eigenvalues tell us how much variance (or information) is present along each eigenvector.

3. A higher eigenvalue means that direction (eigenvector) explains more variance in the data.

4. PCA uses eigenvectors to rotate the coordinate system and align it with the directions of maximum variance.

5. The eigenvectors are orthogonal (at right angles) to each other, ensuring no redundancy.

6. The eigenvalues help in ranking the components — we keep the top ones with the highest eigenvalues.

7. These are computed from the covariance matrix of the standardized data.

8. PCA reduces dimensions by dropping components (eigenvectors) with low eigenvalues.

In short, eigenvectors give direction, and eigenvalues tell importance of those directions in representing the data.

15 How do you decide the number of components to keep in PCA

Ans Deciding the number of components to keep in PCA depends on how much variance (information) you want to retain from the original dataset. Here are common methods to choose:

1. Explained Variance Ratio:
    - PCA provides how much variance each principal component explains.
    - You can add up the variance of the top components until you reach a desired threshold (e.g., 95%).
2. Scree Plot (Elbow Method):
    - Plot the eigenvalues (variance) vs. component number.
    - Look for an "elbow" point where the curve starts to flatten — components after that add little value.
3. Cumulative Variance Plot:
    - Choose the number of components where the cumulative explained variance reaches a threshold like 90–95%.
4. Cross-validation:
    - Test model performance using different numbers of components and select the one with best results.
5. Domain Knowledge:
    - Sometimes, the choice is guided by practical considerations or feature constraints.

16 Can PCA be used for classification

Ans Yes, PCA can be used in classification, but it's important to understand how:

1. PCA is not a classification algorithm itself — it's a dimensionality reduction technique.
2. It can be applied before classification to reduce the number of input features.
3. This helps to remove noise, reduce overfitting, and speed up training.

4. By projecting data into a lower-dimensional space, PCA often makes patterns more separable for classifiers like KNN, SVM, Logistic Regression, etc.

5. However, since PCA is unsupervised, it doesn't consider class labels while reducing dimensions.

6. So, it may not always improve accuracy, especially if important class-separating features are lost.

7. PCA works best when there's redundancy or strong correlation in the input features.

17 What are the limitations of PCA

Ans Here are the main limitations of PCA (Principal Component Analysis):

1. Linear Assumption: PCA only captures linear relationships between variables; it cannot model non-linear patterns.

2. Loss of Interpretability: The new features (principal components) are combinations of original features, making them hard to interpret.

3. Sensitive to Scaling: PCA requires standardized features; otherwise, features with larger scales dominate the principal components.

4. Not Supervised: PCA ignores class labels, so important features for classification may get reduced or removed.

5. Affected by Outliers: PCA is sensitive to outliers, which can distort the direction of principal components.

6. Information Loss: If too few components are chosen, it may result in loss of important information.

7. Assumes Mean and Variance are Sufficient: PCA assumes that the data's structure can be captured through covariance, which isn't always true**.**

18 How do KNN and PCA complement each other

Ans KNN and PCA complement each other well, especially when working with high-dimensional data. Here's how:

1. KNN relies on distance metrics, which become less reliable in high-dimensional spaces due to the curse of dimensionality.

2. PCA reduces dimensions by keeping only the most important variance in the data, helping to remove noise and redundancy.

3. By applying PCA before KNN, you make distance calculations in KNN more meaningful and efficient.

4. PCA helps to speed up KNN by reducing the number of features, which reduces computation time during prediction.

5. It also helps to improve KNN accuracy, especially when the original features are highly correlated or noisy.

6. PCA can help visualize high-dimensional data, making it easier to understand how KNN is classifying points.

19 How does KNN handle missing values in a dataset

Ans KNN does not handle missing values directly, so they must be addressed before applying the algorithm. However, there are ways to handle missing values using KNN-based approaches:

1. Preprocessing Required

- KNN cannot process records with missing data in features used for distance calculation.

- You need to impute (fill) the missing values first.

2. KNN Imputation

- One popular method is KNN imputation, where missing values are filled using the average (for regression) or most frequent value (for classification) from the K nearest neighbors.

- This is more accurate than simple mean/median imputation because it uses data points that are similar.

3. Drop Missing Rows/Columns

- If the amount of missing data is small, you can drop rows or columns with missing values, though this might lead to loss of information.

20 What are the key differences between PCA and Linear Discriminant Analysis (LDA)?

Ans

No.	PCA (Principal Component Analysis)	LDA (Linear Discriminant Analysis)
1.	Unsupervised learning technique	Supervised learning technique
2.	Does not consider class labels	Considers class labels while transforming data
3.	Focuses on maximizing total variance	Focuses on maximizing class separability
4.	Used mainly for dimensionality reduction	Used for both dimensionality reduction and classification
5.	Components are chosen based on eigenvectors of covariance matrix	Components are chosen based on eigenvectors of scatter matrices
6.	Can be applied even when class labels are not known	Requires known class labels
7.	Aims to capture the directions of maximum variance	Aims to maximize the distance between means of different classes

8.	May lose important class information during transformation	Tries to preserve class-discriminative information
9.	Number of components ≤ number of original features	Number of components ≤ (number of classes - 1)
10.	Better for exploratory data analysis and noise reduction	Better for classification tasks where labels are important